leetcode 3

3148二位数组两数差最大值

C

```c
int maxScore(int** grid, int gridSize, int* gridColSize) {
    int min[gridSize][gridColSize[0]],result=INT_MIN;
    min[0][0]=INT_MAX;
    for (int i=1;i<gridSize;i++) {
        min[i][0]=min[i-1][0]<grid[i-1][0]?min[i-1][0]:grid[i-1][0];
    }
    for (int i=1;i<gridColSize[0];i++) {
        min[0][i]=min[0][i-1]<grid[0][i-1]?min[0][i-1]:grid[0][i-1];
    }
    for (int i=1;i<gridSize;i++) {
        for (int j=1;j<gridColSize[0];j++) {
            int u=min[i-1][j]<grid[i-1][j]?min[i-1][j]:grid[i-1][j];
            int l=min[i][j-1]<grid[i][j-1]?min[i][j-1]:grid[i][j-1];
            min[i][j]=u<l?u:l;
        }
    }
    for (int i=0;i<gridSize;i++) {
        for (int j=0;j<gridColSize[0];j++) {
            result=result>grid[i][j]-min[i][j]?result:grid[i][j]-min[i]
[j];
        }
    }
    return result;
}
```

差分：将O(n)操作转换为O(1)操作

2848与车相交的点

C

```c
int numberOfPoints(int** nums, int numsSize, int* numsColSize) {
    int max=0,s=0,result=0;
    for (int i=0;i<numsSize;i++) {
        max=max>nums[i][1]?max:nums[i][1];
    }
    int* diff=calloc(max+2,sizeof(int));
    for (int i=0;i<numsSize;i++) {
```

```
        diff[nums[i][0]]++;
        diff[nums[i][1]+1]--;
    }
    for (int i=0;i<max+1;i++) {
        s+=diff[i];
        result+=s>0;
    }
    return result;
}
```

1109航班预定座位数

C

```
/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
int* corpFlightBookings(int** bookings, int bookingsSize, int*
bookingsColSize, int n, int* returnSize) {
    int* answer=(int*)calloc(n,sizeof(int));
    int* diff=calloc(n+2,sizeof(int));
    int s=0;
    *returnSize=n;
    for (int i=0;i<bookingsSize;i++) {
        diff[bookings[i][0]]+=bookings[i][2];
        diff[bookings[i][1]+1]-=bookings[i][2];
    }
    for (int i=1;i<n+1;i++) {
        s+=diff[i];
        answer[i-1]=s;
    }
    return answer;
}
```

169多数元素（摩尔投票）

C

```
//给定一个大小为 n 的数组 nums ，返回其中的多数元素。多数元素是指在数组中出现次数 大于
⌊ n/2 ⌋ 的元素。
int majorityElement(int* nums, int numsSize) {
    int x=0,vote=0;
    for (int i=0;i<numsSize;i++) {
        if (vote==0) x=nums[i];
```

```
        vote+=nums[i]==x?1:-1;
    }
    return x;
}
//思路为：如果有一个占多数的数字，那么
```

3919下标间移动的最小代价

C

```
//给你一个整数数组 nums，nums 是 严格递增 的。
//对于每个下标 x，设 closest(x) 为使得 abs(nums[x] - nums[y]) 最小化 的 相邻下标。如果两个 相邻 下标的差值相同，则选择 较小 的下标。
//从任意下标 x 出发，你可以通过以下两种方式移动：
//• 移动到任意下标 y，代价为 abs(nums[x] - nums[y])，或者
//• 移动到 closest(x)，代价为 1。
//同时给你一个二维整数数组 queries，其中每个 queries[i] = [li, ri]。
//对于每个查询，计算从下标 li 移动到下标 ri 的 最小总代价。
//返回一个整数数组 ans，其中 ans[i] 是第 i 个查询的答案。
int* minCost(int* nums, int numsSize, int** queries, int queriesSize, int* queriesColSize, int* returnSize) {
    int ltr[numsSize-1], rtl[numsSize-1], preltr[numsSize], prertl[numsSize];
    int closest[numsSize];
    closest[0] = 1;// 计算 closest 下标
    closest[numsSize-1] = numsSize-2;
    for (int i = 1; i < numsSize-1; i++) {
        if (abs(nums[i]-nums[i-1]) <= abs(nums[i+1]-nums[i])) closest[i] = i-1;
        else closest[i] = i+1;
    }
    for (int i = 0; i < numsSize-1; i++) {// 计算 ltr 和 rtl
        if (closest[i] == i+1) ltr[i] = 1;
        else ltr[i] = abs(nums[i+1]-nums[i]);
        if (closest[i+1] == i) rtl[i] = 1;
        else rtl[i] = abs(nums[i+1]-nums[i]);
    }
    preltr[0] = 0;// 前缀和
    for (int i = 1; i < numsSize; i++) preltr[i] = preltr[i-1] + ltr[i-1];
    prertl[numsSize-1] = 0;
    for (int i = numsSize-2; i >= 0; i--) prertl[i] = prertl[i+1] + rtl[i];
    *returnSize = queriesSize;// 处理查询
    int* result = (int*)malloc(sizeof(int) * queriesSize);
    for (int i = 0; i < queriesSize; i++) {
        int l = queries[i][0], r = queries[i][1];
```

```
        if (l == r) result[i] = 0;
        else if (l < r) result[i] = preltr[r] - preltr[l];
        else result[i] = prertl[r] - prertl[l];
    }
    return result;
}
```

995K连续位的最大值

C

```c
//最初的错误代码
int minKBitFlips(int* nums, int numsSize, int k) {
    int act=0,m=0;
    for (int i=0;i<numsSize;i++) {
        if (i<k) act+=(nums[i]+act)%2==1;
        else {
            m+=(nums[i-k]+m)%2==1;
            if (i+k>numsSize && (nums[i]+act-m)%2==0) return -1;
            act+=(nums[i]+act-m)%2==0;
        }
    }
    return act;
}
//DS的正确代码
int minKBitFlips(int* nums, int numsSize, int k) {
    int act = 0, m = 0;
    for (int i = 0; i < numsSize; i++) {
        // 有翻转在当前位置结束
        if (i >= k && nums[i - k] >= 2) {
            m++;                        // 结束一次翻转
            nums[i - k] -= 2;           // 恢复原值（可选，不影响后续）
        }
        int cur = (nums[i] % 2 + act - m) % 2;  // 当前实际值
        if (cur == 0) {                 // 需要翻转
            if (i + k > numsSize) return -1;    // 剩余长度不够翻转
            act++;
            nums[i] += 2;               // 标记此处开始了一次翻转
        }
    }
    return act;
}//原程序错误的原因是，nums[i]+m不能用来判断是否进行过操作！！

```

56合并区间

C

```c
//以数组 intervals 表示若干个区间的集合，其中单个区间为 intervals[i] = [starti, endi] 。请你合并所有重叠的区间，并返回 一个不重叠的区间数组，该数组需恰好覆盖输入中的所有区间 。
//思路为先左端点排序
// 比较函数：按区间左端点升序排序
int cmp(const void* a, const void* b) {
    int* intervalA = *(int**)a;
    int* intervalB = *(int**)b;
    // 题目数据通常不会溢出，直接相减即可
    return intervalA[0] - intervalB[0];
}

int** merge(int** intervals, int intervalsSize, int* intervalsColSize,
            int* returnSize, int** returnColumnSizes) {
    // 处理空输入
    if (intervalsSize == 0) {
        *returnSize = 0;
        *returnColumnSizes = NULL;
        return NULL;
    }

    // 1. 按照左端点排序
    qsort(intervals, intervalsSize, sizeof(int*), cmp);

    // 2. 准备答案容器（最多 intervalsSize 个区间）
    int** ans = (int**)malloc(intervalsSize * sizeof(int*));
    int* colSizes = (int*)malloc(intervalsSize * sizeof(int));
    int count = 0; // 当前已合并的区间个数

    // 3. 遍历排序后的区间
    for (int i = 0; i < intervalsSize; ++i) {
        int left = intervals[i][0];
        int right = intervals[i][1];

        // 如果当前答案为空白，或者当前区间无法与上一个合并
        if (count == 0 || left > ans[count - 1][1]) {
            // 开辟新区间，并拷贝数据
            ans[count] = (int*)malloc(2 * sizeof(int));
            ans[count][0] = left;
            ans[count][1] = right;
            colSizes[count] = 2;   // 每个区间固定为 2 列
            ++count;
        } else {
            // 可以合并，更新右端点最大值
            if (right > ans[count - 1][1]) {
```

```
                ans[count - 1][1] = right;
            }
            // 合并时不增加 count，列宽不变
        }
    }

    // 4. 设置输出参数
    *returnSize = count;
    *returnColumnSizes = colSizes; // 多分配的空间无需回收，调用者只会读取前 count 个
    return ans;
}
```

2536子矩阵全部加一

C

```
int** rangeAddQueries(int n, int** queries, int queriesSize, int* queriesColSize, int* returnSize, int** returnColumnSizes) {// 二维差分
    int** diff = calloc(n + 2, sizeof(int*));
    for (int i = 0; i < n + 2; i++) diff[i] = calloc(n + 2, sizeof(int));
    for (int i = 0; i < queriesSize; i++) {
        int r1 = queries[i][0], c1 = queries[i][1], r2 = queries[i][2], c2 = queries[i][3];
        diff[r1 + 1][c1 + 1]++;
        diff[r1 + 1][c2 + 2]--;
        diff[r2 + 2][c1 + 1]--;
        diff[r2 + 2][c2 + 2]++;
    }
    int** ans = malloc(n * sizeof(int*));// 原地计算 diff 的二维前缀和，然后填入答案
    *returnSize = n;
    *returnColumnSizes = malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        ans[i] = malloc(n * sizeof(int));
        (*returnColumnSizes)[i] = n;
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            diff[i + 1][j + 1] += diff[i + 1][j] + diff[i][j + 1] - diff[i][j];
            ans[i][j] = diff[i + 1][j + 1];
        }
    }
    for (int i = 0; i < n + 2; i++) free(diff[i]);
    free(diff);
```

```
    return ans;
}
```

栈!!!

844比较退格后的字符串

C

```c
bool backspaceCompare(char* s, char* t) {
    int cnts=-1,cntt=-1;
    int lens=strlen(s),lent=strlen(t);
    for (int i=0;i<lens;i++) {
        if (islower(s[i])) s[++cnts]=s[i];
        else cnts-=cnts>-1;//第一位处的#不会退格
    }
    for (int i=0;i<lent;i++) {
        if (islower(t[i])) t[++cntt]=t[i];
        else cntt-=cntt>-1;
    }
    if (cnts>-1) s[cnts+1]='\0';
    if (cntt>-1) t[cntt+1]='\0';//处理最后一位比较好，但是也要注意条件
    if (cnts!=cntt) return false;
    else if (cnts==-1) return true;
    return strncmp(s,t,cntt+1)==0;//strncmp返回值是int。且应该+1比较
}
//strcpy要求第一个是char*，如果直接使用会出现错误
```

1441栈操作构建数组

C

```c
//给你一个数组 target 和一个整数 n。给你一个空栈和两种操作：
//"Push"：将一个整数加到栈顶。"Pop"：从栈顶删除一个整数。同时给定一个范围 [1, n] 中的整数流。
//使用两个栈操作使栈中的数字（从底部到顶部）等于 target。你应该遵循以下规则：
//如果整数流不为空，从流中选取下一个整数并将其推送到栈顶。如果栈不为空，弹出栈顶的整数。如果，在任何时刻，栈中的元素（从底部到顶部）等于 target，则不要从流中读取新的整数，也不要对栈进行更多操作。
//请返回遵循上述规则构建 target 所用的操作序列。如果存在多个合法答案，返回 任一 即可。
/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
char** buildArray(int* target, int targetSize, int n, int* returnSize) {
```

```
    int cnt=0,size=target[targetSize-1]>n?target[targetSize-1]:n,pos=0;
    char** result=(char**)calloc(size*2,sizeof(char*));//注意此处char需要加上*
    for (int i=0;i<size*2;i++) result[i]=(char*)calloc(5,sizeof(char));//注意还有\0，需要多一位
    for (int i=0;i<n && cnt<targetSize;i++) {//注意是从0还是1开始循环
        if (i+1==target[cnt]) {
            strcpy(result[pos],"Push");
            cnt++;
            pos++;
        }
        else {
            strcpy(result[pos],"Push");
            strcpy(result[pos+1],"Pop");
            pos+=2;
        }
    }
    *returnSize=pos;
    return result;
}
//一个数字要被跳过，需要先入栈而后出栈
//返回数组的大小不超过2*size
//cnt用于：计数到了target数组的哪里
//pos用于：计数result数组中有多少位置了
```

946验证栈序列

C

```
//给定 pushed 和 popped 两个序列，每个序列中的 值都不重复，只有当它们可能是在最初空栈上进行的推入 push 和弹出 pop 操作序列的结果时，返回 true；否则，返回 false 。
bool validateStackSequences(int* pushed, int pushedSize, int* popped, int poppedSize) {
    int push=-1,pop=0;
    for (int i=0;i<pushedSize;i++) {
        pushed[++push]=pushed[i];
        while (push>=0 && pushed[push]==popped[pop]) {//这里不要用if，用while
            push--;
            pop++;
        }
    }
    return push==-1;
}
```

3412返回字符串的镜像匹配分数

C++

```cpp
//给你一个字符串 s。
//英文字母中每个字母的 镜像 定义为反转字母表之后对应位置上的字母。例如，'a' 的镜像是 'z'，'y' 的镜像是 'b'。
//最初，字符串 s 中的所有字符都 未标记 。字符串 s 的初始分数为 0 ，你需要对其执行以下过程：
//从左到右遍历字符串。对于每个下标 i ，找到距离最近的 未标记 下标 j，下标 j 需要满足 j < i 且 s[j] 是 s[i] 的镜像。然后 标记 下标 i 和 j，总分加上 i - j 的值。
//如果对于下标 i，不存在满足条件的下标 j，则跳过该下标，继续处理下一个下标，不需要进行标记。
//返回最终的总分。
class Solution {
public:
    long long calculateScore(string s) {
        stack<int> stk[26];
        long long ans(0);
        for (int i=0;i<s.size();i++) {
            int c=s[i]-'a';
            if (!stk[25-c].empty()) {
                ans+=i-stk[25-c].top();
                stk[25-c].pop();
            }
            else stk[c].push(i);
        }
        return ans;
    }
};
```

C++

```cpp
char* clearStars(char* s) {
    int len=strlen(s),low=26,total=0,t=0;//low表示已有的最小字符，total表示处理后的总长度
    int** mem=(int**)calloc(26,sizeof(int*));//用于计数每个字母出现的位置
    int hash[26];//用于记录每个字母出现的次数
    for (int i=0;i<26;i++) hash[i]=-1;
    for (int i=0;i<26;i++) mem[i]=(int*)calloc(len,sizeof(int));
    for (int i=0;i<len;i++) {
        if (islower(s[i])) {//是字母的，记录出现位置，次数加一，验证最小值
            mem[s[i]-'a'][++hash[s[i]-'a']]=i;
            low=s[i]-'a'<low?s[i]-'a':low;
        }
```

```
        else {//在不越界的情况下减一，必须及时更新low
            if (low!=26 && hash[low]>=0) {
                hash[low]--;
                if (low<26 && hash[low]==-1) {
                    while (low<26 && hash[low]==-1) low++;//
                }
            }
        }
    }
    for (int i=0;i<26;i++) total+=hash[i]+1;//记录总长度
    char* result=(char*)malloc((total+1)*sizeof(char));
    char* mock=(char*)calloc(len,sizeof(char));
    for (int i=0;i<26;i++) {
        if (hash[i]>=0) {
            for (int j=0;j<=hash[i];j++) mock[mem[i][j]]='a'+i;
        }
    }
    for (int i=0;i<len;i++) {
        if (islower(mock[i]) || mock[i]=='*') result[t++]=mock[i];
    }
    result[total]='\0';
    for (int i=0;i<26;i++) free(mem[i]);
    free(mem);
    free(mock);
    return result;
}
```

2434打印字典序最小的串

C

```
char* robotWithString(char* s) {
    //思路：先进后出的栈，出栈条件为比后续的最小值都要小
    int len=strlen(s);
    char* suf=(char*)malloc((len+1)*sizeof(char));
    suf[len]='z';
    for (int i=len-1;i>=0;i--) suf[i]=suf[i+1]<s[i]?suf[i+1]:s[i];
    char *ans=s,*stack=suf;
    int top=-1,pos=0;
    for (int i=0;i<len;i++) {
        stack[++top]=s[i];
        while (top>=0 && suf[i+1]>=stack[top]) ans[pos++]=stack[top--];
    }
    ans[len]='\0';
    free(suf);
    return ans;
```

```
}
```

C++

```cpp
class Solution {
public:
    string robotWithString(string s) {
        int len=s.size();//string内置了哪些函数?
        vector<char> suf(len+1);//vector是什么? 如何初始化? 如何进行操作?
        suf[len]='z';
        for (int i=len-1;i>=0;i--) suf[i]=suf[i+1]<s[i]?suf[i+1]:s[i];
        string ans;
        stack<char> stk;//stack是什么? 是stl嘛? 如何初始化及操作?
        for (int i=0;i<len;i++) {
            stk.push(s[i]);
            while (!stk.empty() && suf[i+1]>=stk.top()) {
                ans+=stk.top();//对于一个string的+=是什么意思?
                stk.pop();
            }
        }
        return ans;
    }
};
```

2696删除串间相邻AB或CD后字符串的长度

C++

```cpp
class Solution {
public:
    int minLength(string s) {
        stack<char> stack;
        for (char c:s) {//注意这种循环方式
            if (!stack.empty() && (stack.top()=='A' || stack.top()=='C') && c-stack.top()==1) {
                stack.pop();
            }
            else stack.push(c);
        }
        return stack.size();//注意常见函数，以及加括号
    }
};
```

C

```c
int minLength(char* s) {
    int len=strlen(s),top=-1;
    char* stack=(char*)malloc(len*sizeof(char));
    for (int i=0;i<len;i++) {
        stack[++top]=s[i];
        if (top>0 && (stack[top]=='D' || stack[top]=='B') && stack[top]-stack[top-1]==1) top-=2;
    }
    return top+1;
}
```

1047删除相邻相同字母后的结果

C++

```cpp
class Solution {
public:
    string removeDuplicates(string s) {
        string ans;
        for (char c:s) {
            if (!ans.empty() && c==ans.back()) ans.pop_back();
            else ans.push_back(c);
        }
        return ans;
    }
};
```

3043最长公共矩阵长度

C++

```cpp
class Solution {
public:
    int longestCommonPrefix(vector<int>& arr1, vector<int>& arr2) {
        unordered_set<int> hash;
        int max=0;
        for (int i:arr1) {
            for (;i>0;i/=10) hash.insert(i);
        }
        for (int i:arr2) {
            while (!hash.contains(i) && i>0) i/=10;
```

```
            max=max>i?max:i;
        }
        return max?to_string(max).length():0;//此处为将int转换为string的方法
    }
};
```

1209删除相连相同字符后的字符串

C++

```
//给你一个字符串 s，「k 倍重复项删除操作」将会从 s 中选择 k 个相邻且相等的字母，并删除它们，使被删去的字符串的左侧和右侧连在一起。
//你需要对 s 重复进行无限次这样的删除操作，直到无法继续为止。在执行完所有删除操作后，返回最终得到的字符串。本题答案保证唯一。
class Solution {
public:
    string removeDuplicates(string s, int k) {
        stack<int> cnt;//只定义关于相同数字数量的栈，直接在原字符串上操作
        for (int i=0;s[i];i++) {
            if (i==0 || s[i]!=s[i-1]) cnt.push(1);//开始计数一段新的连续相同数字
            else if (++cnt.top()==k) {//将增加计数操作与判断结合
                cnt.pop();
                s.erase(i-k+1,k);//又是不认识的stl函数
                i-=k;//s总量减少了k，计数的i就要减少k
            }
        }
        return s;
    }
};
```

3703移除连续k组匹配括号后的字符串

C++

```
class Solution {
public:
    string removeSubstring(string s, int k) {
        // 用 vector 模拟栈，每个元素是 (字符, 该字符连续出现的次数)
        vector<pair<char, int>> st;

        for (char b : s) {                      // 遍历原字符串每个字符
            // 1. 更新连续计数
            if (!st.empty() && st.back().first == b) {
```

```
                st.back().second++;             // 与栈顶字符相同，计数+1
            } else {
                st.emplace_back(b, 1);          // 不同，压入新字符，计数为1
            }

            // 2. 尝试消除：当前字符是 ')' 且 栈顶恰好有 k 个 ')'，且它下面有至少 k 个 '('
            if (b == ')' && st.size() > 1 && st.back().second == k && st[st.size()-2].second >= k) {
                st.pop_back();                  // 移除这一整段 k 个 ')'
                st.back().second -= k;          // 从下面的 '(' 段中减去 k
                if (st.back().second == 0) {    // 如果 '(' 被减光了，也移除
                    st.pop_back();
                }
            }
        }

        // 3. 根据栈中剩余内容构建最终字符串
        string ans;
        for (auto& p : st) {
            ans += string(p.second, p.first);  // 重复 p.second 次字符 p.first
        }
        return ans;
    }
};
```